```
puts "You gave #{ARGV.size} arguments"
p ARGV
```

When we run it with arguments, we can see that they get passed in:

```
$ ruby cmd_line.rb ant bee cat dog
You gave 4 arguments
["ant", "bee", "cat", "dog"]
```

Often, the arguments to a program are the names of files that you want to process. In this case, you can use a second technique: the variable ARGF is a special kind of I/O object that acts like all the contents of all the files whose names are passed on the command line (or standard input if you don't pass any filenames). We'll look at that some more in ARGF, on page 238.

Commenting Ruby

Ruby has two ways of adding comments to source code, one of which you will use, and one of which you will almost certainly not use. The common one is the # symbol—anything after that symbol until the end of the line is a comment and is ignored by the interpreter. If the next line continues the comment, it needs its own # symbol.

Ruby also has a vary rarely used multiline comment, where the first line starts with =begin and everything is a comment until the code reaches =end. Both the =begin and =end must be at the very beginning of the line, they cannot be indented.

While we did just say that Ruby ignores comments, Ruby uses a very small number of "magic comments" for configuration options on a per-file basis. These comments have the form of # directive: value and must appear in the file before the first line of actual Ruby code.

The most commonly used magic comment is # frozen_string_literal: true. If this directive is true, then every string literal that does not have an interpolation inside it will automatically be frozen, as though freeze was called on it.

You might also see an # encoding: VALUE directive, which specifies the encoding for string and regular expression literals inside that particular file. Ruby also has a # warn_indent: BOOLEAN flag that will throw code warnings if a file's indentation is mismatched. There's an experimental directive called # sharable_constant_value: which affects how values are shared using the Ractor multithreading tools.

What's Next

We finished our lightning-fast tour of some of the basic features of Ruby. We took a look at objects, methods, strings, containers, and regular expressions; saw some simple control structures; and we looked at some rather nifty iterators. We hope this chapter has given you enough ammunition to be able to attack the rest of this book.

It's time to move on and move up—up to a higher level. Next, we'll be looking at classes and objects, things that are at the same time both the highest-level constructs in Ruby and the essential underpinnings of the entire language.